How much data is sufficient to learn high-performing algorithms?

Generalization guarantees for data-driven algorithm design

Ellen Vitercik
Carnegie Mellon University

Columbia Theory Seminar

Nina Balcan

Dan DeBlasio

Travis Dick

Carl Kingsford

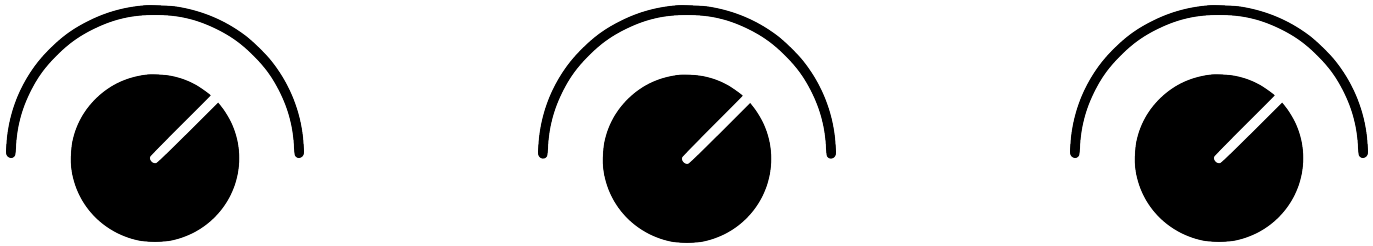
Tuomas Sandholm

Data-driven algorithm design

Algorithms often have **many tunable parameters**

Significant impact on runtime, solution quality, …

Hand-tuning is **time-consuming**, **tedious**, and **error prone**



Data-driven algorithm design

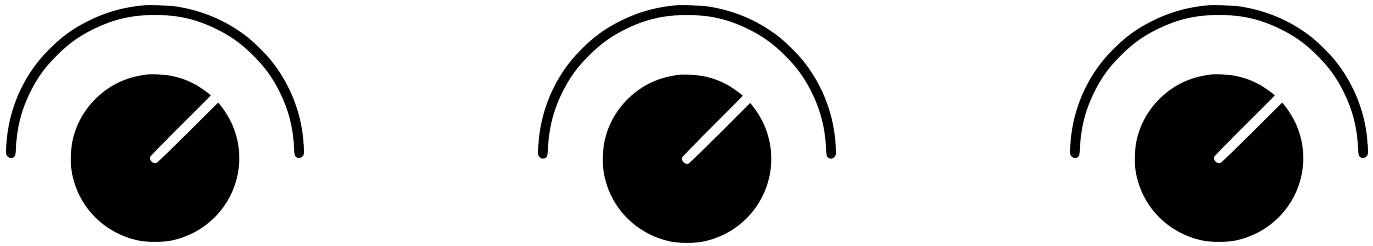
Goal: Automate algorithm configuration via machine learning

Algorithmically find good parameter settings

using a **set of "typical" inputs** from application at hand

Training set

Parameter setting should – **ideally** – be good on future inputs



Example: Sequence alignment

Goal: Line up pairs of strings

Applications: Biology, natural language processing, etc.

Sequence alignment algorithms

Input: Two sequences S and S'

$$S = \text{A C T G}$$
$$S' = \text{G T C A}$$

Output: Alignment of S and S'

Gap

A – – C T G
– G T C A –

Insertion/deletion (*indel*)

Match

Mismatch

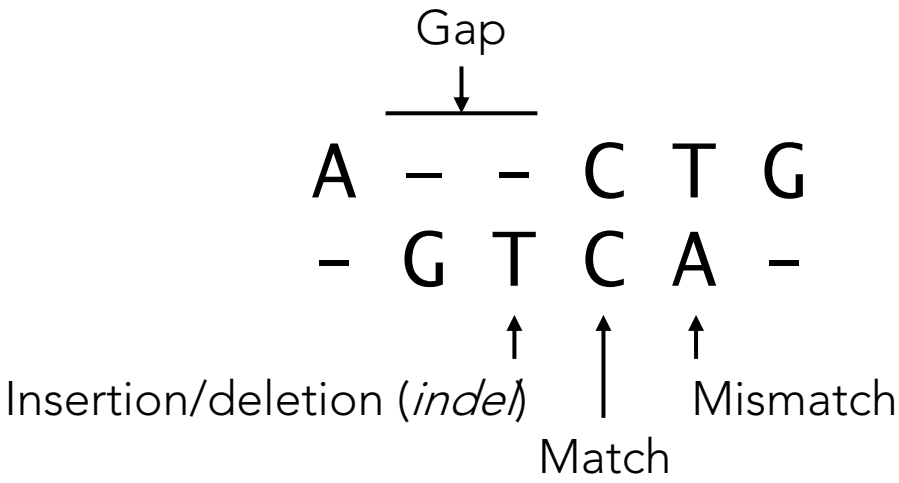
Sequence alignment algorithms

Standard algorithm with parameters $\rho_1, \rho_2, \rho_3 \geq 0$:

Return alignment maximizing:

$$(\#\text{ matches}) - \rho_1 \cdot (\#\text{ mismatches}) - \rho_2 \cdot (\#\text{ indels}) - \rho_3 \cdot (\#\text{ gaps})$$

$$S = \text{A C T G}$$
$$S' = \text{G T C A}$$



Sequence alignment algorithms

Can sometimes access **ground-truth, reference** alignment

E.g., in computational biology: Bahr et al., Nucleic Acids Res.'01; Raghava et al., BMC Bioinformatics '03; Edgar, Nucleic Acids Res.'04; Walle et al., Bioinformatics'04

Requires extensive manual alignments

…rather just run parameterized algorithm

How to tune algorithm's parameters?

*"There is **considerable disagreement** among molecular biologists about the **correct choice**"* [Gusfield et al. '94]

```
A - - C T G
- G T C A -
```

Sequence alignment algorithms

```
-GRTCPKPDDLPFSTVVP-LKTFYEPGEEITYSCKPGYVSRGGMRKFICPLTGLWPINTLKCTP
E-VKCPFPSRPDNGFVNYPAKPTLYYKDKATFGCHDGYSLDGP-EEIECTKLGNWSAMPSC-KA
```

Ground-truth alignment of protein sequences

Sequence alignment algorithms

```
-GRTCPKPDDLPFSTVVP-LKTFYEPGEEITYSCKPGYVSRGGMRKFICPLTGLWPINTLKCTP
E-VKCPFPSRPDNGFVNYPAKPTLYYKDKATFGCHDGYSLDGP-EEIECTKLGNWSAMPSC-KA
```

Ground-truth alignment of protein sequences

```
GRTCP---KPDDLPFSTVVPLKTFYEPGEEITYSCKPGYVSRGGMRKFICPLTGLWPINTLKCTP
EVKCPFPSRPDN-GFVNYPAKPTLYYK-DKATFGCHDGY-SLDGPEEIECTKLGNWS-AMPSCKA
```

Alignment by algorithm with **poorly-tuned** parameters

Sequence alignment algorithms

```
-GRTCPKPDDLPFSTVVP-LKTFYEPGEEITYSCKPGYVSRGGMRKFICPLTGLWPINTLKCTP
E-VKCPFPSRPDNGFVNYPAKPTLYYKDKATFGCHDGYSLDGP-EEIECTKLGNWSAMPSC-KA
```

Ground-truth alignment of protein sequences

```
GRTCP---KPDDLPFSTVVPLKTFYEPGEEITYSCKPGYVSRGGMRKFICPLTGLWPINTLKCTP
EVKCPFPSRPDN-GFVNYPAKPTLYYK-DKATFGCHDGY-SLDGPEEIECTKLGNWS-AMPSCKA
```

Alignment by algorithm with **poorly-tuned** parameters

```
GRTCPKPDDLPFSTV-VPLKTFYEPGEEITYSCKPGYVSRGGMRKFICPLTGLWPINTLKCTP
EVKCPFPSRPDNGFVNYPAKPTLYYKDKATFGCHDGY-SLDGPEEIECTKLGNWSA-MPSCKA
```

Alignment by algorithm with **well-tuned** parameters

Automated parameter tuning procedure

1. Fix parameterized algorithm
2. Receive training set T of "typical" inputs

Sequence S_1
Sequence S_1'
Reference alignment A_1

Sequence S_2
Sequence S_2'
Reference alignment A_2

• • •

3. Find parameters with good performance on average over T

Runtime, solution quality, etc.

Automated parameter tuning procedure

1. Fix parameterized algorithm
2. Receive training set T of "typical" inputs

Sequence S_1
Sequence S_1'
Reference alignment A_1

Sequence S_2
Sequence S_2'
Reference alignment A_2

• • •

3. Find parameters with good performance on average over T

Output alignment is close to reference alignment

Automated parameter tuning procedure

1. Fix parameterized algorithm
2. Receive training set T of "typical" inputs

Sequence S_1
Sequence S_1'
Reference alignment A_1

Sequence S_2
Sequence S_2'
Reference alignment A_2

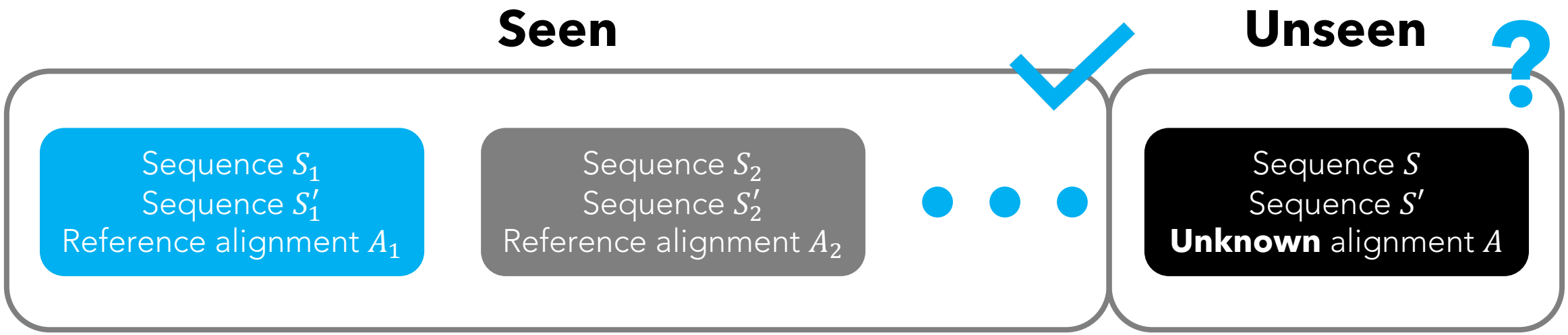
• • •

3. Find parameters with good performance on average over T

Key question (focus of talk):

Will those parameters have good **future** performance?

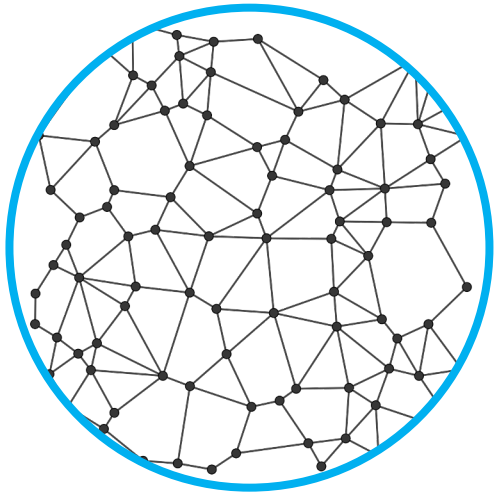
Automated parameter tuning procedure



Key question (focus of talk):

Will those parameters have good **future** performance?

Automated parameter tuning

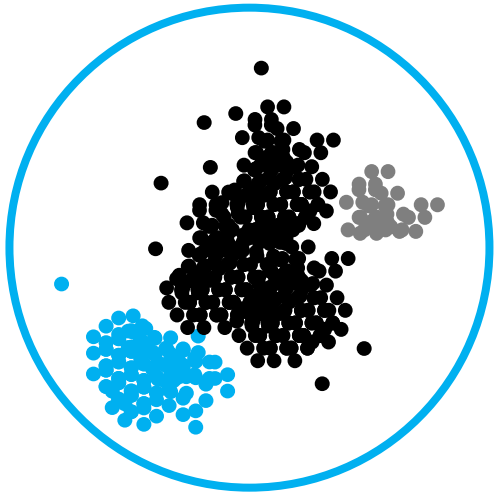


Model applies in **many** settings, including:

Constraint satisfaction problems, e.g.:

Horvitz, Ruan, Gomes, Krautz, Selman, Chickering	UAI'01
Nudelman, Leyton-Brown, Hoos, Devkar, Shoham	CP'04
Sayag, Fine, Mansour	STACS'06
Hutter, Hamadi, Hoos, Leyton-Brown	CP'06
Xu, Hutter, Leyton-Brown	CP'07
Xu, Hutter, Hoos, Leyton-Brown	JAIR'08
Xu, Hoos, Leyton-Brown	AAAI'10
Kleinberg, Leyton-Brown, Lucier	IJCAI'17
Balcan, Dick, Sandholm, **Vitercik**	ICML'18
Weisz, György, Szepesvári	ICML '18, '19
Kleinberg, Leyton-Brown, Lucier, Graham	NeurIPS'19

Automated parameter tuning

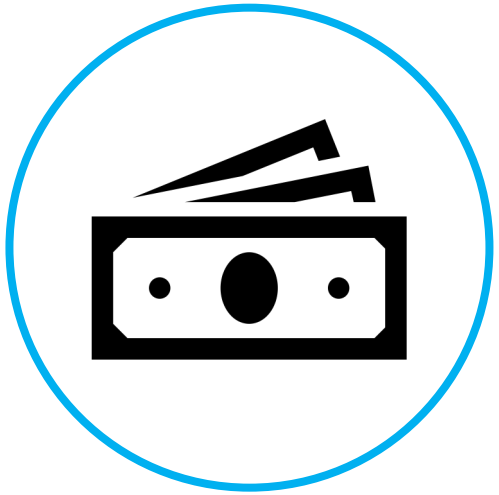


Model applies in **many** settings, including:

Clustering, e.g.:

Balcan, Nagarajan, **Vitercik**, White	COLT'17
Balcan, Dick, White	NeurIPS'18
Garg, Kalai	NeurIPS'18
Balcan, Dick, Lang	ICLR'20

Automated parameter tuning

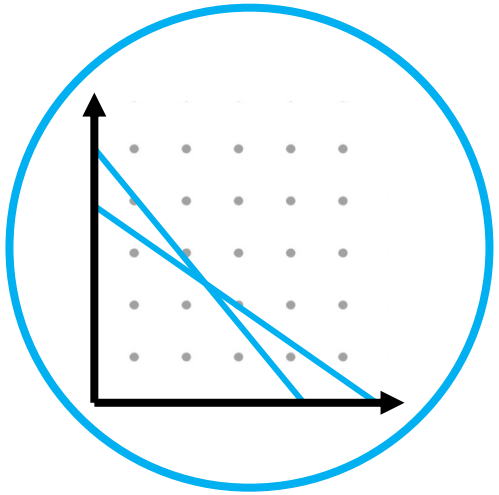


Model applies in **many** settings, including:

Sales algorithms (mechanisms), e.g.:

Likhodedov, Sandholm	AAAI '04, '05
Balcan, Blum, Hartline, Mansour	FOCS'05, JCSS'08
Elkind	SODA'07
Cole, Roughgarden	STOC'14
Mohri, Muñoz	ICML'14
Huang, Mansour, Roughgarden	EC'15
Morgenstern, Roughgarden	NeurIPS'15, COLT'16
Balcan, Sandholm, **Vitercik**	NeurIPS'16
Balcan, Sandholm, **Vitercik**	EC'18
Balcan, Dick, **Vitercik**	FOCS'18
Balcan, Sandholm, **Vitercik**	EC'19

Automated parameter tuning



Model applies in **many** settings, including:

Integer and linear programming, e.g.:

Leyton-Brown, Nudelman, Andrew, McFadden, Shoham	IJCAI'03, CP'03
Hutter, Hoos, Leyton-Brown, Stützle	JAIR'09
Hutter, Hoos, Leyton-Brown	LION'11, AIJ'14
Sandholm	Handbook of Market Design'13
He, Daume, Eisner	NeurIPS'14
Khalil, Le Bodic, Song, Nemhauser, Dilkina	AAAI'16
Balcan, Nagarajan, **Vitercik**, White	COLT'17
Balcan, Dick, Sandholm, **Vitercik**	ICML'18
Balcan, Dick, **Vitercik**	FOCS'18
Balcan, Sandholm, **Vitercik**	AAAI'20
Balcan, Sandholm, **Vitercik**	ICML'20

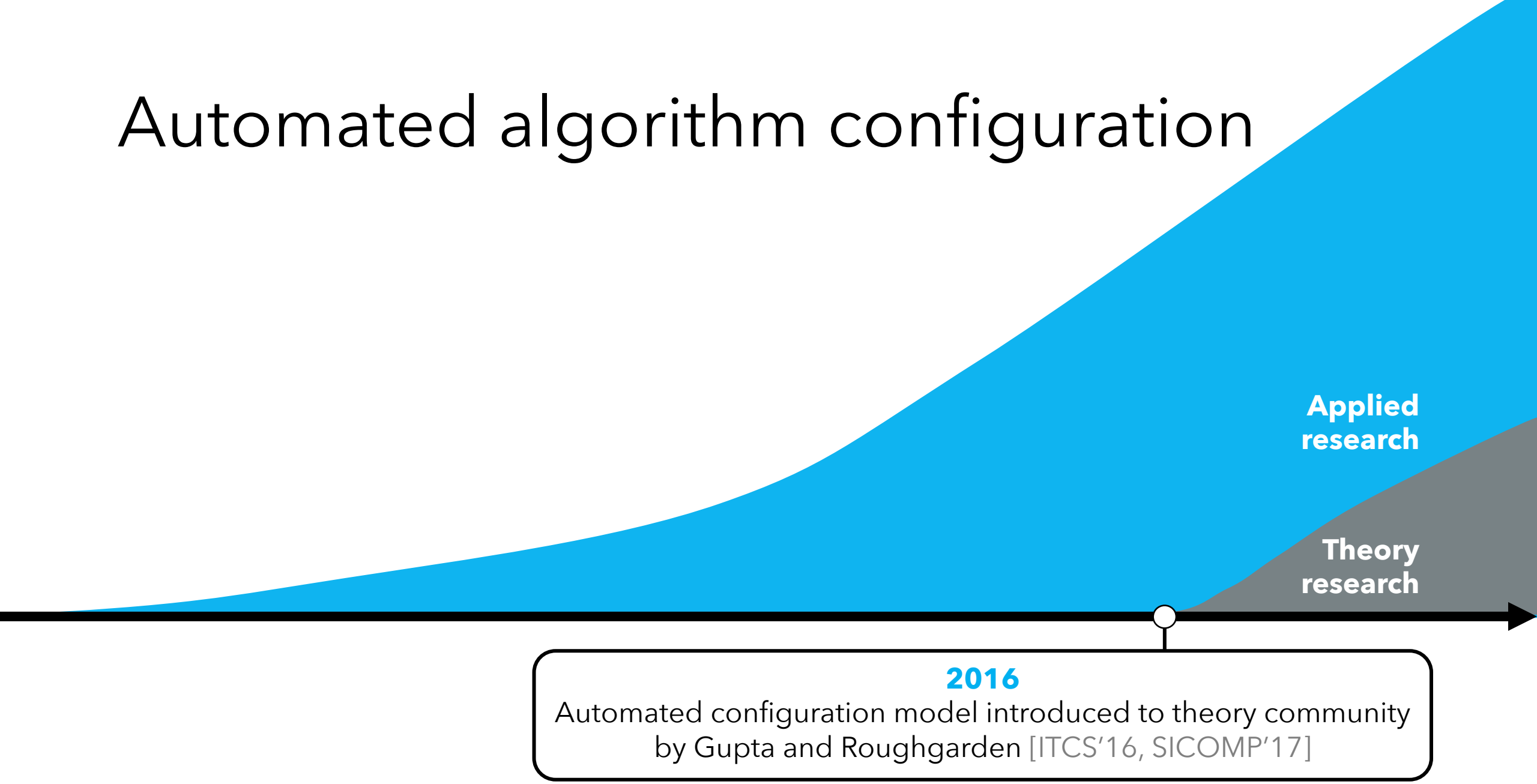
Automated parameter tuning

Model applies in **many** settings, including:

Computational biology, e.g.:

Majoros, Salzberg	Bioinformatics'04
Chikhi, Medvedev	Bioinformatics'13
May, Tamura, Noble	J. of Proteome Research'17
DeBlasio, Kececioglu	Springer'18
DeBlasio, Kim, Kingsford	WBC@ICML'19
Balcan, DeBlasio, Dick, Kingsford, Sandholm, **Vitercik**	'20

Automated algorithm configuration



This talk: Main result

Key question (focus of talk):

Good performance on **average** over **training set** implies good **future** performance?

Answer this question for any parameterized algorithm where:

Performance is **piecewise-structured** function of parameters

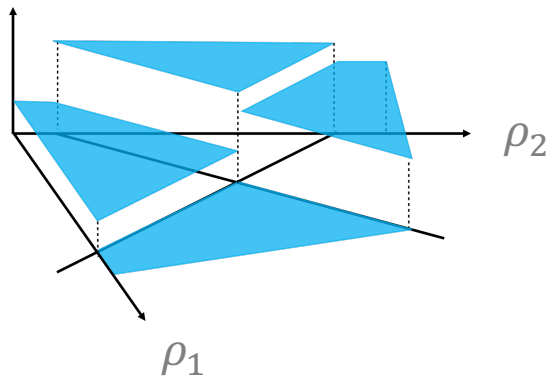
Piecewise constant, linear, quadratic, …

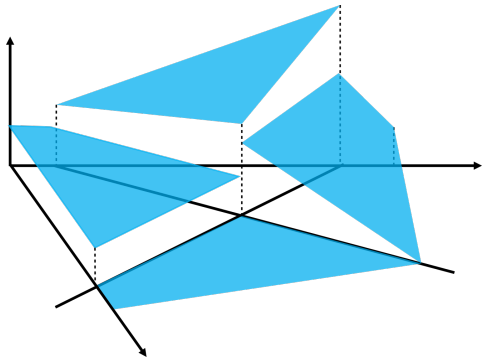
This talk: Main result

Performance is **piecewise-structured** function of parameters

Piecewise constant, linear, quadratic, …

Algorithmic performance on fixed input



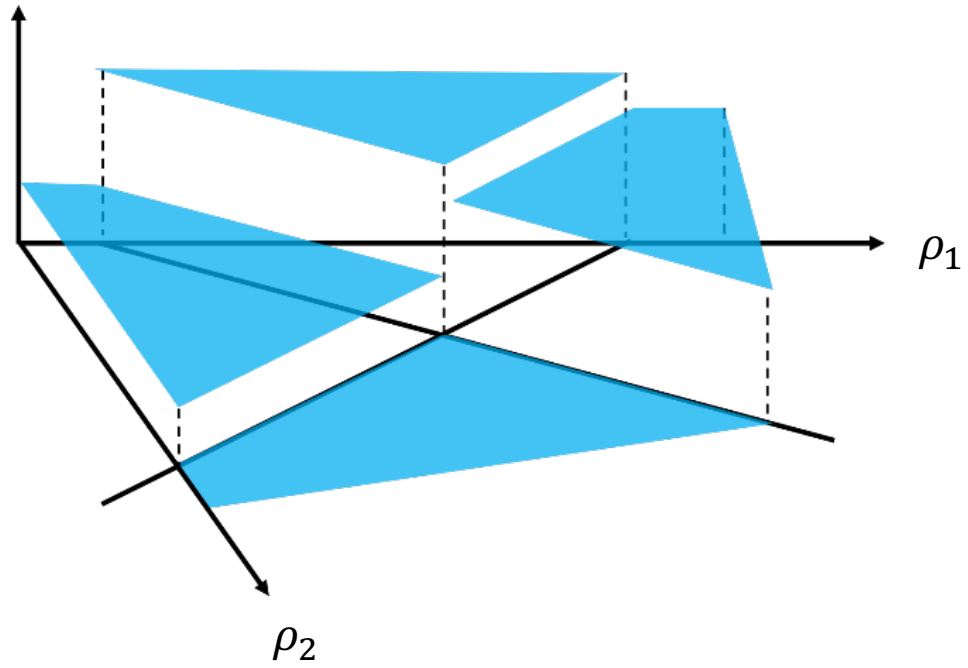


• • •

Piecewise constant **Piecewise linear** **Piecewise …**

Example: Sequence alignment

Distance between **algorithm's output** given S, S'
and **ground-truth** alignment is p-wise constant

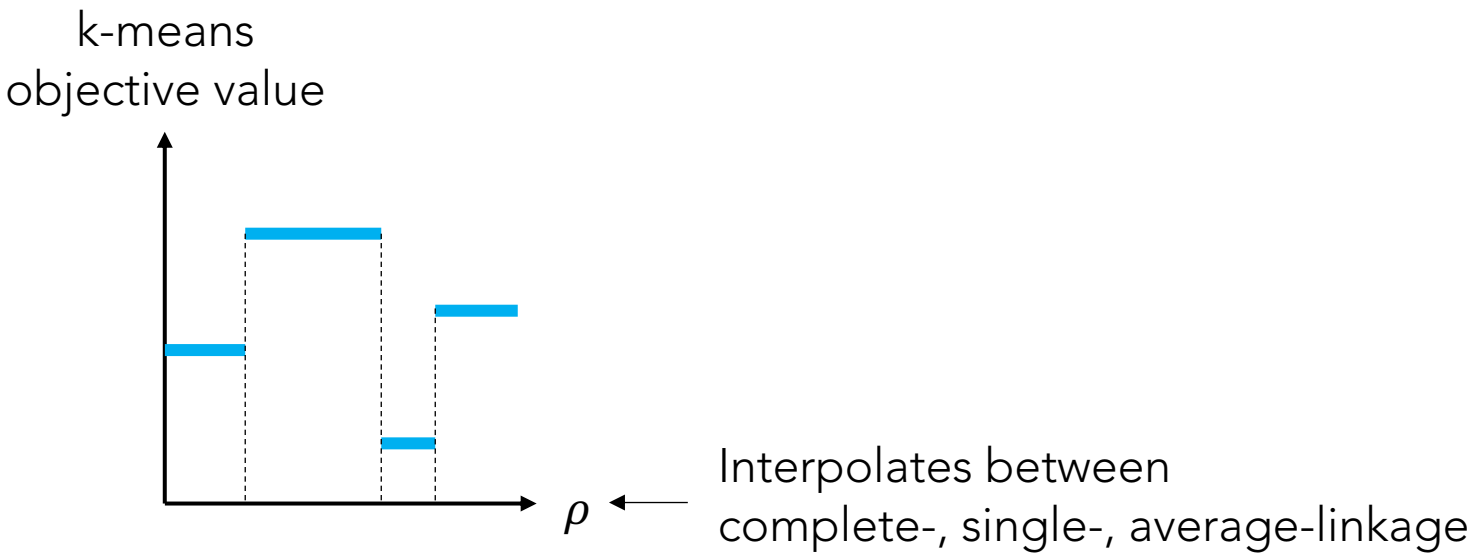


Main results

Exhibit this piecewise structure in **many** diverse settings:

Clustering

Balcan, Nagarajan, **Vitercik**, White, COLT'17



Main results

Exhibit this piecewise structure in **many** diverse settings:

Clustering

Balcan, Nagarajan, **Vitercik**, White, COLT'17

Sales algorithms (mechanisms):

Balcan, Sandholm, **Vitercik**, NeurIPS'16; Balcan, Sandholm, **Vitercik**, EC'18;
Balcan, DeBlasio, Dick, Kingsford, Sandholm, **Vitercik**, '19

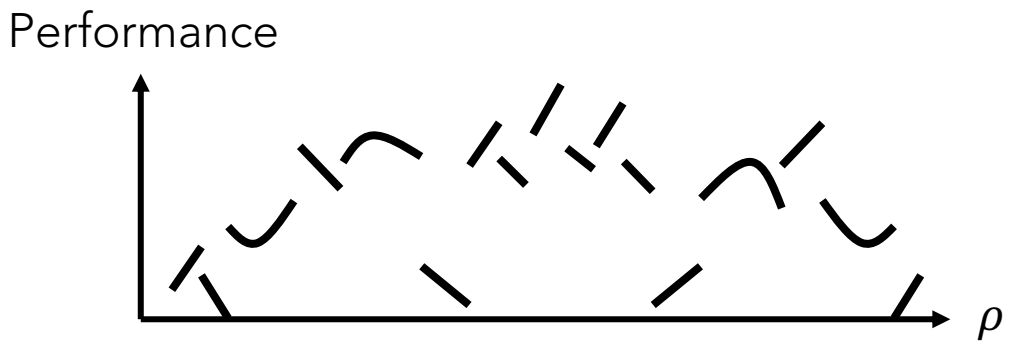
Integer programming algorithms:

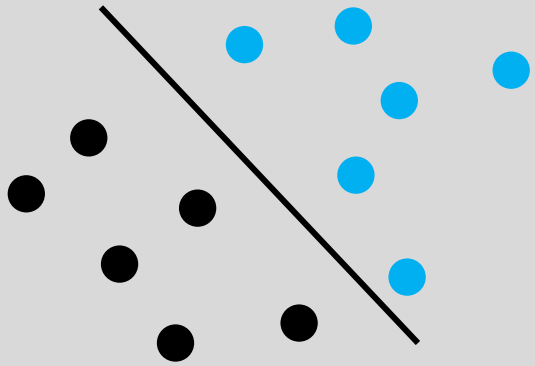
Balcan, Nagarajan, **Vitercik**, White, COLT'17; Balcan, Dick, Sandholm, **Vitercik**, ICML'18

Primary challenge:

Algorithmic performance is a **volatile** function of parameters

Complex connection between parameters and performance





For well-understood functions in machine learning theory:

Simple connection between function parameters and value

Outline

1. Introduction
2. **Model and problem formulation**
3. Our guarantees
4. Conclusion and future directions

Model

$\mathbb{R}^d$: Set of all parameters

$\mathcal{X}$: Set of all inputs

Example: Sequence alignment

$\mathbb{R}^3$: Set of alignment algorithm parameters

$\mathcal{X}$: Set of sequence pairs

$$\begin{aligned} S &= \mathrm{A\ C\ T\ G} \\ S' &= \mathrm{G\ T\ C\ A} \end{aligned}$$

One sequence pair $x = (S, S') \in \mathcal{X}$

Algorithmic performance

$u_{\boldsymbol{\rho}}(x)$ = utility of algorithm parameterized by $\boldsymbol{\rho} \in \mathbb{R}^d$ on input x

E.g., runtime, solution quality, distance to ground truth, …

Assume $u_{\boldsymbol{\rho}}(x) \in [-1,1]$

Can be generalized to $u_{\boldsymbol{\rho}}(x) \in [-H, H]$

Algorithmic performance

$u_{\boldsymbol{\rho}}(x)$ = distance between algorithm's output and ground-truth

S = A C T G
S' = G T C A

A – – C T G
– G T C A –

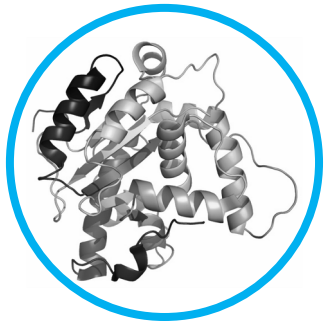
One sequence pair $x = (S, S') \in \mathcal{X}$

Model

Standard assumption: Unknown distribution $\mathcal{D}$ over inputs

Distribution models specific application domain at hand

E.g., distribution over pairs of DNA strands



E.g., distribution over pairs of protein sequences

Generalization bounds

Key question: For any parameter setting $\boldsymbol{\rho}$,
is **average** utility on training set close to **expected** utility?

Formally: Given samples $x_1, \ldots, x_N \sim \mathcal{D}$, for any $\boldsymbol{\rho}$,

$$\left| \underbrace{\frac{1}{N} \sum_{i=1}^{N} u_{\boldsymbol{\rho}}(x_i)}_{\text{Empirical average utility}} - \mathbb{E}_{x \sim \mathcal{D}}\left[u_{\boldsymbol{\rho}}(x)\right] \right| \leq \textbf{?}$$

Generalization bounds

Key question: For any parameter setting $\boldsymbol{\rho}$,
is **average** utility on training set close to **expected** utility?

Formally: Given samples $x_1, \ldots, x_N \sim \mathcal{D}$, for any $\boldsymbol{\rho}$,

$$\left| \frac{1}{N} \sum_{i=1}^{N} u_{\boldsymbol{\rho}}(x_i) - \underbrace{\mathbb{E}_{x \sim \mathcal{D}}\left[u_{\boldsymbol{\rho}}(x)\right]}_{\textbf{Expected utility}} \right| \leq \textbf{?}$$

Outline

1. Introduction
2. Model and problem formulation
3. Our guarantees
   a. **Example of piecewise-structured utility function**
   b. Piecewise-structured functions more formally
   c. Main theorem
   d. Application: Sequence alignment
4. Conclusion and future directions

Sequence alignment algorithms

Standard algorithm with parameters $\rho_1, \rho_2, \rho_3 \geq 0$:

Return alignment maximizing:

$$(\#\text{ matches}) - \rho_1 \cdot (\#\text{ mismatches}) - \rho_2 \cdot (\#\text{ indels}) - \rho_3 \cdot (\#\text{ gaps})$$

$$S = \text{A C T G}$$
$$S' = \text{G T C A}$$

Gap

A – – C T G

– G T C A –

Mismatch

Match

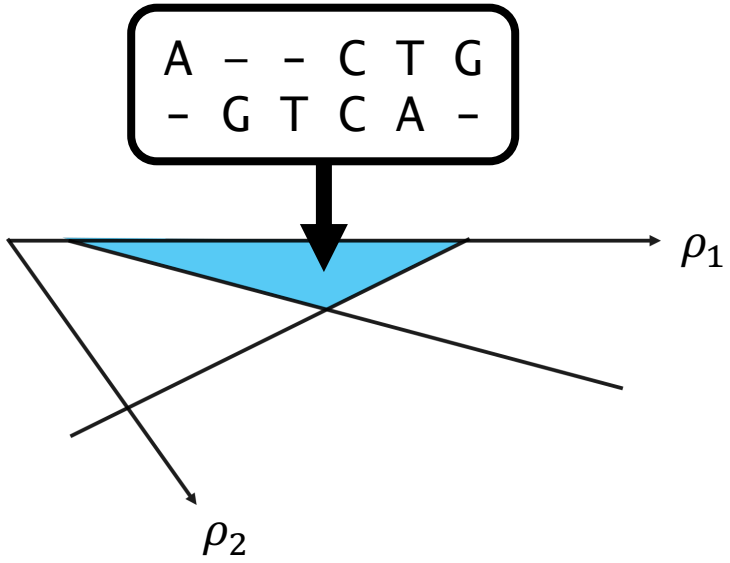
Insertion/deletion (*indel*)

Sequence alignment algorithms

Lemma:

For any pair S, S', there's a small partition of $\mathbb{R}^3$ s.t. in any region, algorithm's output is fixed across all parameters in region

S = A C T G
S' = G T C A

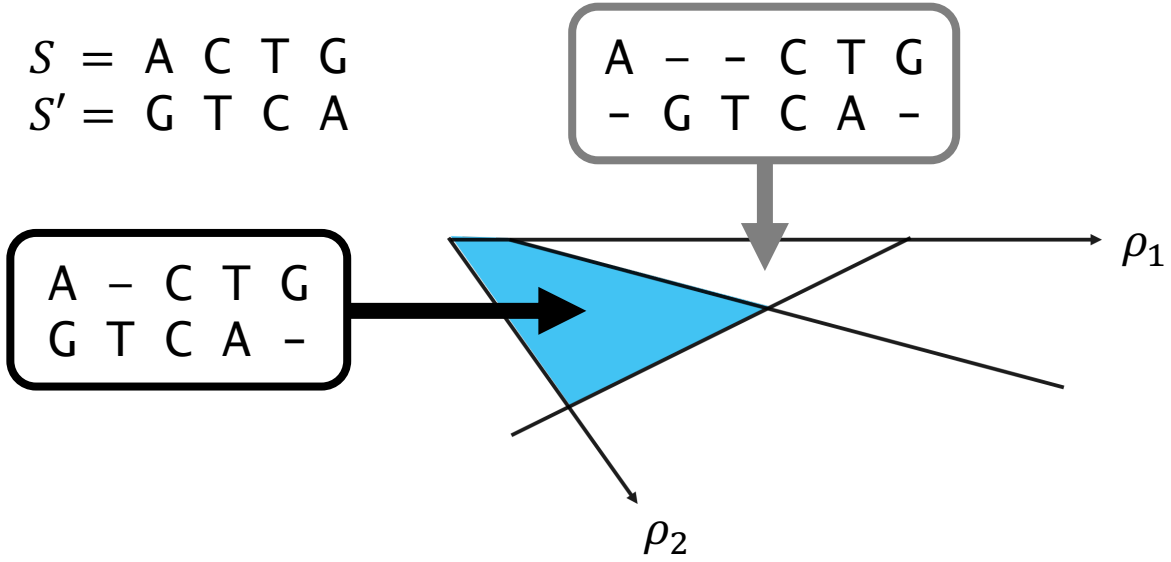


Gusfield et al., Algorithmica '94; Fernández-Baca et al., J. of Discrete Alg. '04

Sequence alignment algorithms

Lemma:

For any pair S, S', there's a small partition of $\mathbb{R}^3$ s.t. in any region, algorithm's output is fixed across all parameters in region



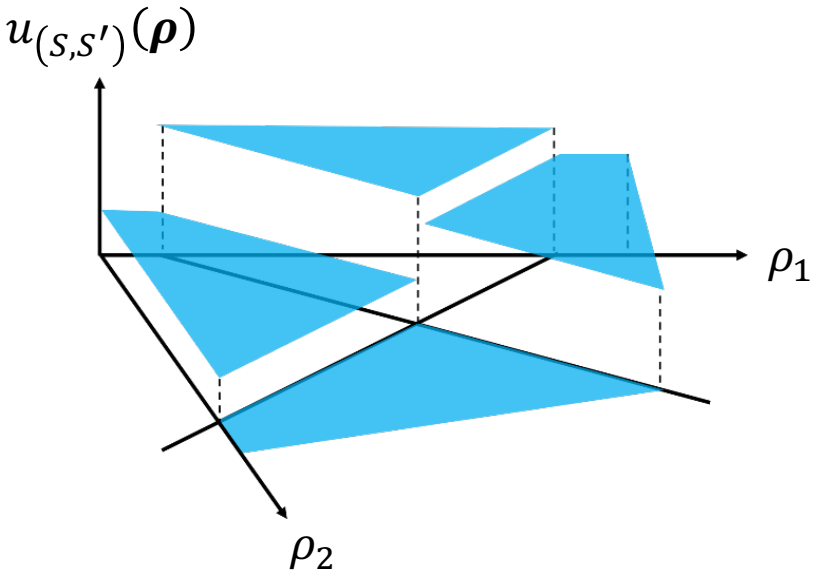
Gusfield et al., Algorithmica '94; Fernández-Baca et al., J. of Discrete Alg. '04

Piecewise-constant utility function

Corollary:

Utility is piecewise constant function of parameters

Distance between algorithm's output and ground-truth alignment



Piecewise-structured functions

Clustering algorithms:

Piecewise constant function of parameter

Balcan, Nagarajan, **Vitercik**, White, COLT'17

Sales algorithms (mechanisms):

Piecewise linear

Balcan, Sandholm, **Vitercik**, NeurIPS'16; Balcan, Sandholm, **Vitercik**, EC'18;
Balcan, DeBlasio, Dick, Kingsford, Sandholm, **Vitercik**, '19

Integer programming algorithms:

Piecewise constant and piecewise inverse-quadratic

Balcan, Nagarajan, **Vitercik**, White, COLT'17; Balcan, Dick, Sandholm, **Vitercik**, ICML'18

Outline

1. Introduction
2. Model and problem formulation
3. Our guarantees
   a. Example of piecewise-structured utility function
   b. Piecewise-structured functions more formally
   c. Main theorem
   d. Application: Sequence alignment
4. Conclusion and future directions

Primal & dual classes

$u_{\boldsymbol{\rho}}(x)$ = utility of algorithm parameterized by $\boldsymbol{\rho} \in \mathbb{R}^d$ on input x

$\mathcal{U} \ = \{u_{\boldsymbol{\rho}}: \mathcal{X} \to \mathbb{R} \mid \boldsymbol{\rho} \in \mathbb{R}^d\}$ **"Primal" function class**

Typically, prove guarantees by bounding ***complexity*** of $\mathcal{U}$

VC dimension, pseudo-dimension, Rademacher complexity, …

Primal & dual classes

$u_{\boldsymbol{\rho}}(x)$ = utility of algorithm parameterized by $\boldsymbol{\rho} \in \mathbb{R}^d$ on input x

$\mathcal{U} \;\; = \{u_{\boldsymbol{\rho}} : \mathcal{X} \to \mathbb{R} \mid \boldsymbol{\rho} \in \mathbb{R}^d\}$ **"Primal" function class**

Typically, prove guarantees by bounding ***complexity*** of $\mathcal{U}$

Challenge: $\mathcal{U}$ is gnarly

E.g., in sequence alignment:

- Each domain element is a pair of sequences
- Unclear how to plot or visualize functions $u_{\boldsymbol{\rho}}$
- No obvious notions of Lipschitzness or smoothness to rely on

Primal & dual classes

$u_{\boldsymbol{\rho}}(x)$ = utility of algorithm parameterized by $\boldsymbol{\rho} \in \mathbb{R}^d$ on input x
$\mathcal{U} \ = \{u_{\boldsymbol{\rho}}: \mathcal{X} \to \mathbb{R} \mid \boldsymbol{\rho} \in \mathbb{R}^d\}$ **"Primal" function class**

$u_x^*(\boldsymbol{\rho})$ = utility as function of parameters
$u_x^*(\boldsymbol{\rho}) = u_{\boldsymbol{\rho}}(x)$
$\mathcal{U}^* = \{u_x^*: \mathbb{R}^d \to \mathbb{R} \mid x \in \mathcal{X}\}$ **"Dual" function class**

- Dual functions have simple, Euclidean domain
- Often have ample structure can use to bound complexity of $\mathcal{U}$

Piecewise-structured functions

Clustering algorithms

Selling algorithms

Integer programming algorithms

String alignment algorithms

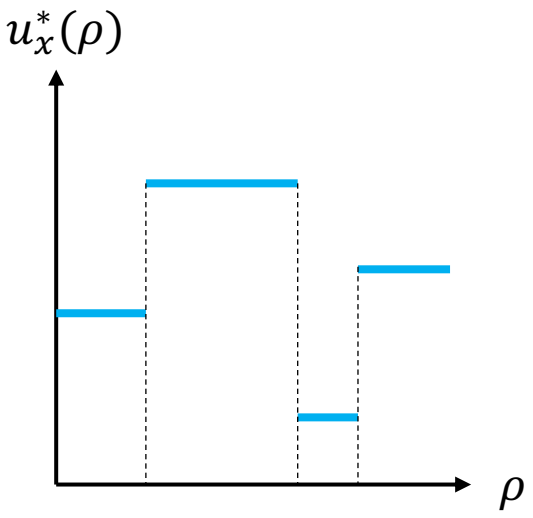
Dual functions $u_x^*: \mathbb{R}^d \to \mathbb{R}$ are **piecewise-structured**

Outline

1. Introduction
2. Model and problem formulation
3. Our guarantees
   a. Example of piecewise-structured utility function
   b. Piecewise-structured functions more formally
   c. Main theorem
   d. Application: Sequence alignment
4. Conclusion and future directions

Warmup: 1-dimensional parameters

For every input x, $u_x^*: \mathbb{R} \to \mathbb{R}$ is p-wise constant with $\leq k$ pieces.



Warmup: 1-dimensional parameters

For every input x, $u_x^*: \mathbb{R} \to \mathbb{R}$ is p-wise constant with $\leq k$ pieces.

Theorem:

Training set of size $\tilde{O}\left(\frac{\log k}{\epsilon^2}\right)$ implies that WHP, for all $\forall \rho$,

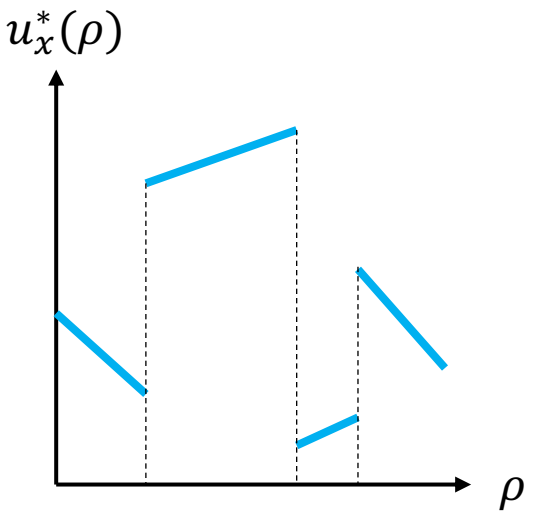
average utility over training set is ϵ-close to **expected** utility

$$\left| \frac{1}{N} \sum_{i=1}^{N} u_\rho(x_i) - \mathbb{E}_{x \sim \mathcal{D}}\left[u_\rho(x)\right] \right| \leq \epsilon$$

Warmup: 1-dimensional parameters

For every input x, $u_x^*: \mathbb{R} \to \mathbb{R}$ is p-wise ~~constant~~ with $\leq k$ pieces.

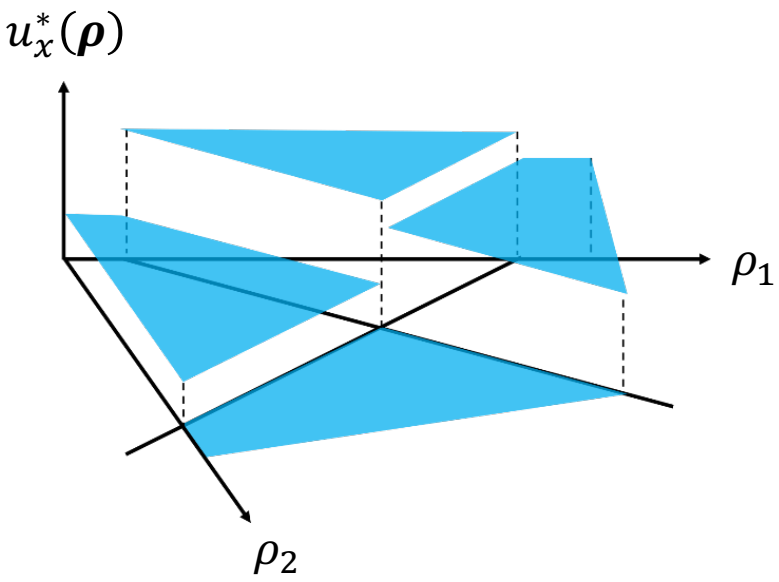
linear



Warmup: 1-dimensional parameters

For every input x, $u_x^*: \mathbb{R} \to \mathbb{R}$ is p-wise ~~constant~~ with $\leq k$ pieces.

???

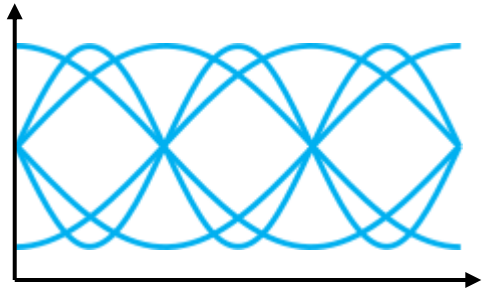


Multiple parameters?

Intrinsic complexity

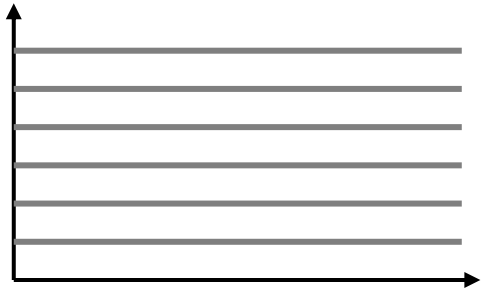
"Intrinsic complexity" of function class $\mathcal{G}$

- Measures how well functions in $\mathcal{G}$ fit complex patterns
- Specific ways to quantify "intrinsic complexity":
  - VC dimension
  - Pseudo-dimension



More complex

Less complex

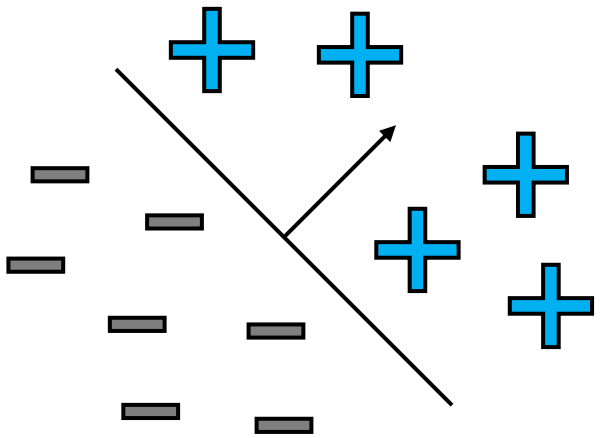


VC dimension

Complexity measure for binary-valued function classes $\mathcal{F}$

(Classes of functions $f: \mathcal{Y} \to \{-1,1\}$)

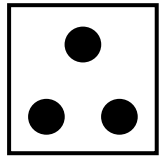
E.g., linear separators



VC dimension of $\mathcal{F}$

Size of the largest set $\mathcal{S} \subseteq \mathcal{Y}$
that can be labeled in all $2^{|\mathcal{S}|}$ ways by functions in $\mathcal{F}$

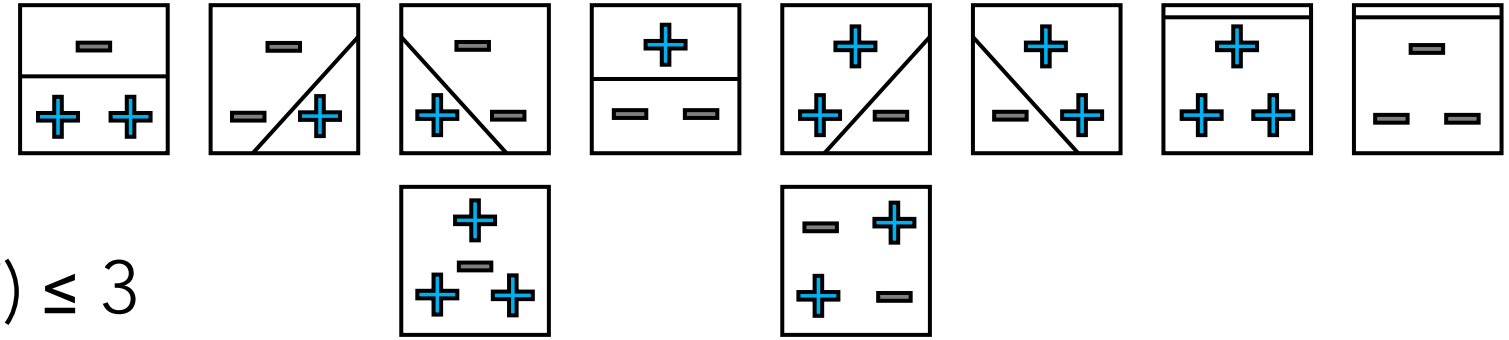
Example: $\mathcal{F}$ = Linear separators in $\mathbb{R}^2$ VCdim($\mathcal{F}$) ≥ 3



VC dimension of $\mathcal{F}$

Size of the largest set $\mathcal{S} \subseteq \mathcal{Y}$
that can be labeled in all $2^{|\mathcal{S}|}$ ways by functions in $\mathcal{F}$

Example: $\mathcal{F}$ = Linear separators in $\mathbb{R}^2$ VCdim($\mathcal{F}$) ≥ 3



VCdim($\mathcal{F}$) ≤ 3

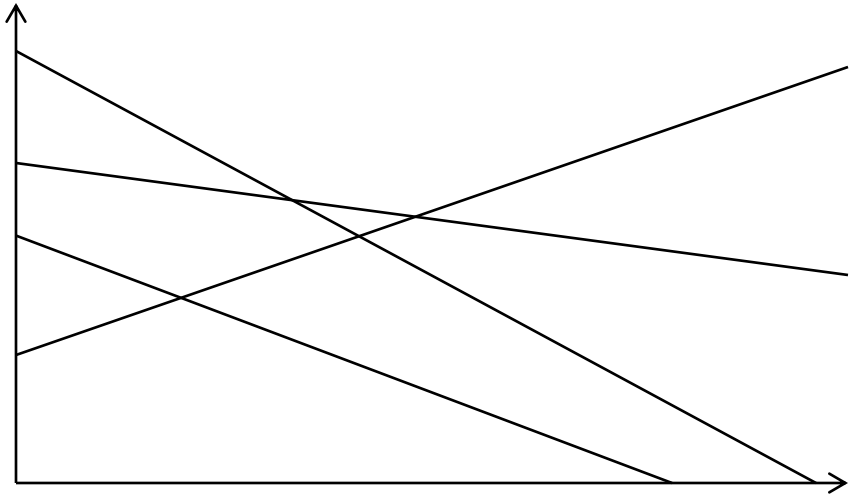
VCdim({Linear separators in $\mathbb{R}^d$}) $= d + 1$

Pseudo-dimension

Complexity measure for real-valued function classes $\mathcal{G}$

(Classes of functions $g: \mathcal{Y} \to \mathbb{R}$)

E.g., affine functions



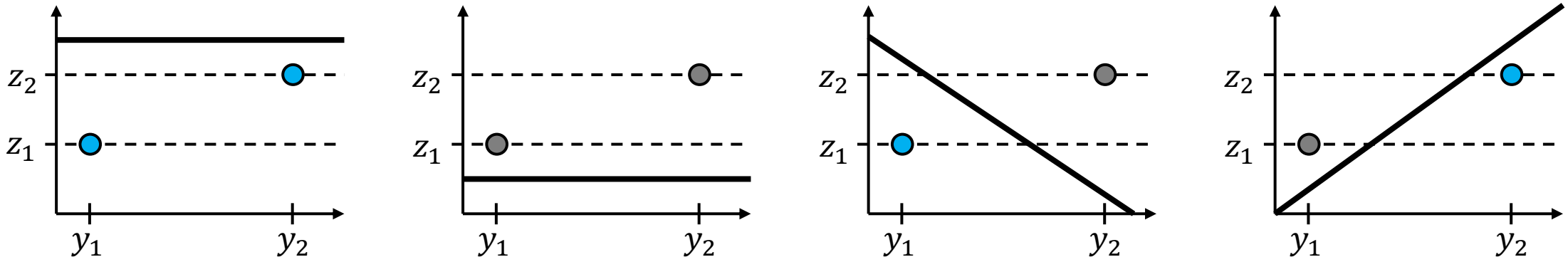
Pseudo-dimension of $\mathcal{G}$

Size of the largest set $\{y_1, \dots, y_N\} \subseteq \mathcal{Y}$ s.t.:

for some *targets* $z_1, \dots, z_N \in \mathbb{R}$,

all 2^N above/below patterns achieved by functions in $\mathcal{G}$

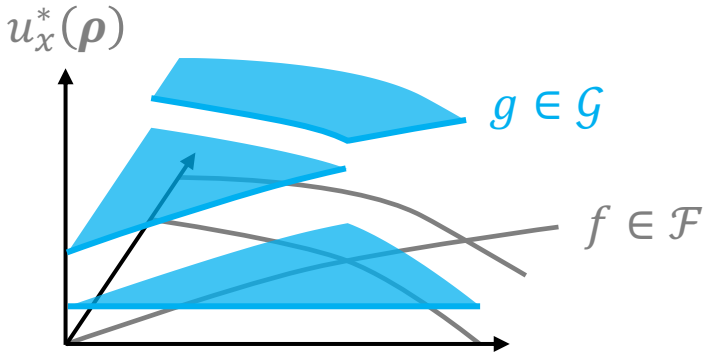
Example: $\mathcal{G}$ = Affine functions in $\mathbb{R}$ Pdim($\mathcal{G}$) ≥ 2



Main result (informal)

Boundary functions $f_1, \ldots, f_k \in \mathcal{F}$ partition $\mathbb{R}^d$ s.t. in each region, $u_x^*(\boldsymbol{\rho}) = g(\boldsymbol{\rho})$ for some $g \in \mathcal{G}$.

Training set of size $\tilde{O}\left(\frac{1}{\epsilon^2}\left(\mathrm{VCdim}(\mathcal{F}^*) + \mathrm{Pdim}(\mathcal{G}^*)\right)\log k\right)$ implies WHP $\forall \boldsymbol{\rho}$, **avg** utility over training set is ϵ-close to **exp** utility



Main result (informal)

Boundary functions $f_1, \ldots, f_k \in \mathcal{F}$ partition $\mathbb{R}^d$ s.t. in each region, $u_x^*(\boldsymbol{\rho}) = g(\boldsymbol{\rho})$ for some $g \in \mathcal{G}$.

Training set of size $\tilde{O}\left(\frac{1}{\epsilon^2}\left(\mathrm{VCdim}(\mathcal{F}^*) + \mathrm{Pdim}(\mathcal{G}^*)\right)\log k\right)$ implies WHP $\forall \boldsymbol{\rho}$, **avg** utility over training set is ϵ-close to **exp** utility

$\mathcal{F}, \mathcal{G}$ are typically very well structured

- $\mathcal{G}$ = set of all **constant** functions $\Rightarrow \mathrm{Pdim}(\mathcal{G}^*) = O(1)$
- $\mathcal{G}$ = set of all **linear** functions in $\mathbb{R}^d$ $\Rightarrow \mathrm{Pdim}(\mathcal{G}^*) = O(d)$

Main result (informal)

Boundary functions $f_1, \ldots, f_k \in \mathcal{F}$ partition $\mathbb{R}^d$ s.t. in each region,

$u_x^*(\boldsymbol{\rho}) = g(\boldsymbol{\rho})$ for some $g \in \mathcal{G}$.

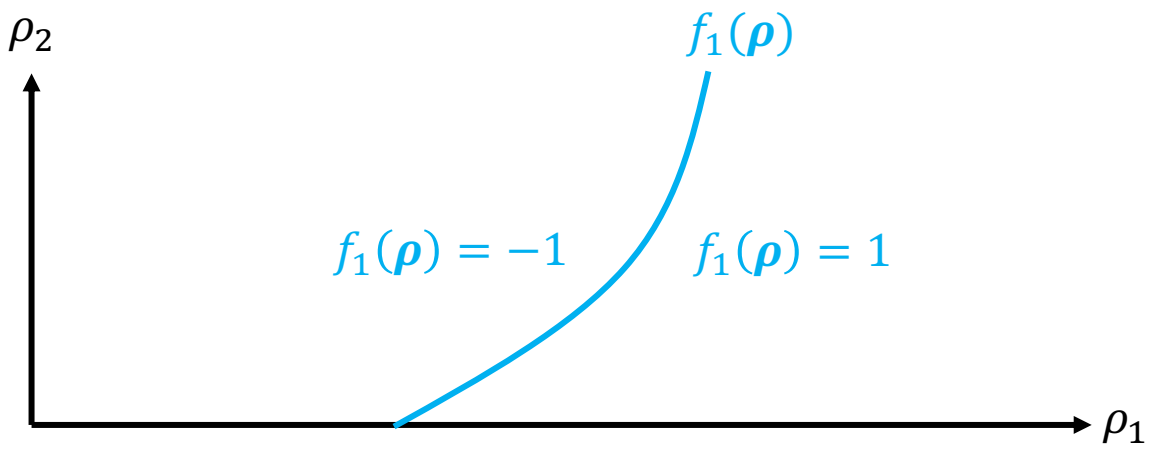
Theorem:

$$\text{Pdim}(\mathcal{U}) = \tilde{O}\big((\text{VCdim}(\mathcal{F}^*) + \text{Pdim}(\mathcal{G}^*)) \log k\big)$$

Primal function class $\mathcal{U} = \{u_{\boldsymbol{\rho}} \mid \boldsymbol{\rho} \in \mathbb{R}^d\}$

Key lemma

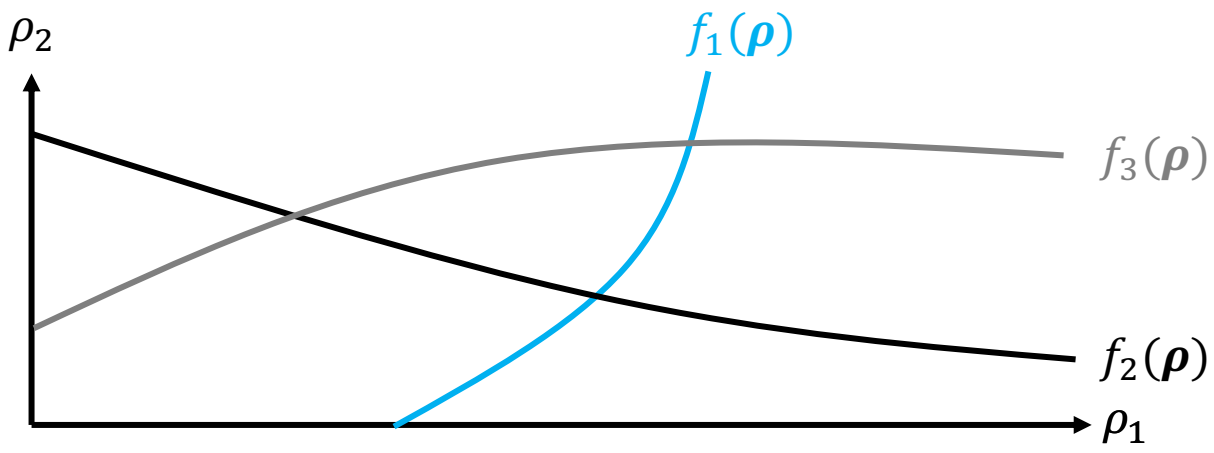
Each boundary function $f: \mathbb{R}^d \to \{-1,1\}$ splits $\mathbb{R}^d$ into 2 regions



Key lemma

Given D boundaries, how many sign patterns do they make?

$$\left|\left\{\begin{pmatrix} f_1(\boldsymbol{\rho}) \\ \vdots \\ f_D(\boldsymbol{\rho}) \end{pmatrix} : \boldsymbol{\rho} \in \mathbb{R}^d\right\}\right| \leq ?$$



Key lemma

Given D boundaries, how many sign patterns do they make?

$$\left|\left\{\begin{pmatrix} f_1(\boldsymbol{\rho}) \\ \vdots \\ f_D(\boldsymbol{\rho}) \end{pmatrix} : \boldsymbol{\rho} \in \mathbb{R}^d\right\}\right| \leq ?$$

Note: Sauer's lemma tells us that for any D points $\boldsymbol{\rho}_1, \ldots, \boldsymbol{\rho}_D \in \mathbb{R}^d$

$$\left|\left\{\begin{pmatrix} f(\boldsymbol{\rho}_1) \\ \vdots \\ f(\boldsymbol{\rho}_D) \end{pmatrix} : f \in \mathcal{F}\right\}\right| \leq (eD)^{\mathrm{VCdim}(\mathcal{F})}$$

This is where transitioning to the dual comes in handy!

Key lemma

Given D boundaries, how many sign patterns do they make?

$$\left|\left\{\begin{pmatrix} f_1(\boldsymbol{\rho}) \\ \vdots \\ f_D(\boldsymbol{\rho}) \end{pmatrix} : \boldsymbol{\rho} \in \mathbb{R}^d\right\}\right| \leq (eD)^{\mathrm{VCdim}(\mathcal{F}^*)}$$

Proof ideas

For any problem instances $x_1, \ldots, x_N$ and *targets* $z_1, \ldots, z_N \in \mathbb{R}$,

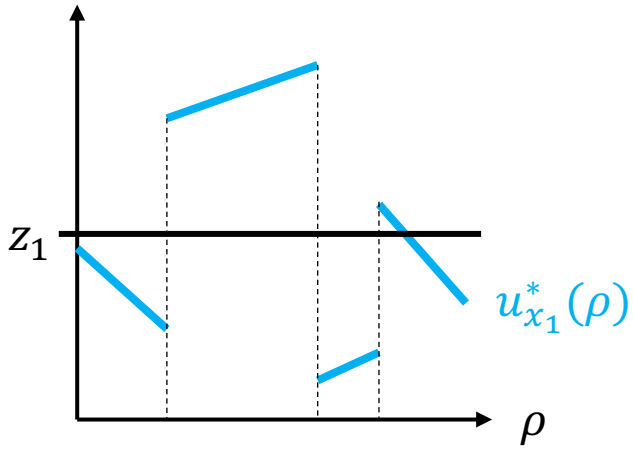
$$\left| \left\{ \begin{pmatrix} \operatorname{sgn}\big(u_{\boldsymbol{\rho}}(x_1) - z_1\big) \\ \vdots \\ \operatorname{sgn}\big(u_{\boldsymbol{\rho}}(x_N) - z_N\big) \end{pmatrix} : \boldsymbol{\rho} \in \mathbb{R}^d \right\} \right| \leq ?$$

Switching to the dual functions,

$$\left| \left\{ \begin{pmatrix} \operatorname{sgn}\big(u^*_{x_1}(\boldsymbol{\rho}) - z_1\big) \\ \vdots \\ \operatorname{sgn}\big(u^*_{x_N}(\boldsymbol{\rho}) - z_N\big) \end{pmatrix} : \boldsymbol{\rho} \in \mathbb{R}^d \right\} \right| \leq ?$$

Proof ideas

$$\left|\left\{\begin{pmatrix} \operatorname{sgn}\left(u^*_{x_1}(\boldsymbol{\rho}) - z_1\right) \\ \vdots \\ \operatorname{sgn}\left(u^*_{x_N}(\boldsymbol{\rho}) - z_N\right) \end{pmatrix} : \boldsymbol{\rho} \in \mathbb{R}^d \right\}\right| \leq ?$$

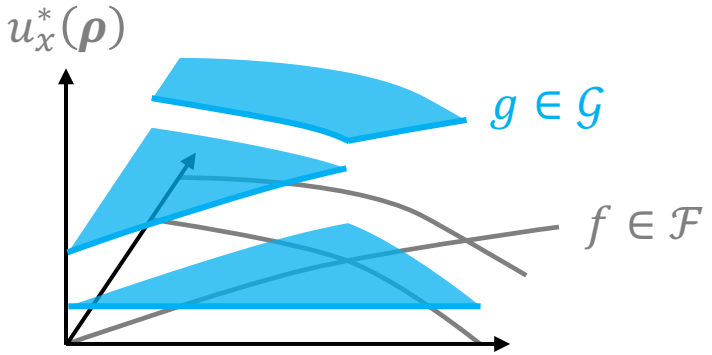


Proof ideas

$$\left|\left\{\begin{pmatrix} \operatorname{sgn}\left(u^*_{x_1}(\boldsymbol{\rho}) - z_1\right) \\ \vdots \\ \operatorname{sgn}\left(u^*_{x_N}(\boldsymbol{\rho}) - z_N\right) \end{pmatrix} : \boldsymbol{\rho} \in \mathbb{R}^d \right\}\right| \leq ?$$

The duals $u^*_{x_1}, \ldots, u^*_{x_N}$ correspond to Nk boundary functions in $\mathcal{F}$

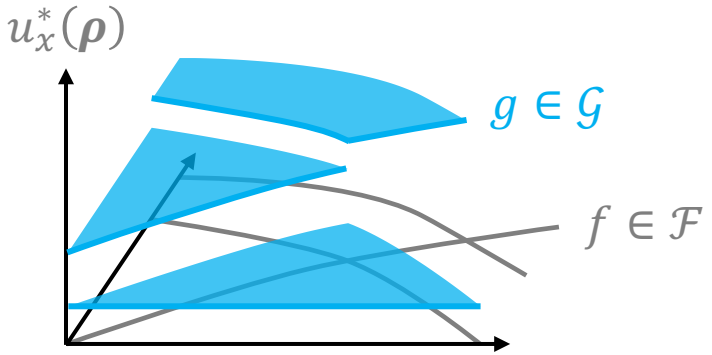
How many regions $R_1, \ldots, R_M$ in $\mathbb{R}^d$? $M \leq (eNk)^{\mathrm{VCdim}(\mathcal{F}^*)}$



Proof ideas

$$\left|\left\{\begin{pmatrix} \mathrm{sgn}\left(u^*_{x_1}(\boldsymbol{\rho}) - z_1\right) \\ \vdots \\ \mathrm{sgn}\left(u^*_{x_N}(\boldsymbol{\rho}) - z_N\right) \end{pmatrix} : \boldsymbol{\rho} \in R_j \right\}\right| \leq ?$$

$\forall \boldsymbol{\rho} \in R_j$, duals are simultaneously structured: $u^*_{x_i}(\boldsymbol{\rho}) = g_i(\boldsymbol{\rho}), \forall i$



Proof ideas

$$\left|\left\{\begin{pmatrix} \operatorname{sgn}\left(u^*_{x_1}(\boldsymbol{\rho}) - z_1\right) \\ \vdots \\ \operatorname{sgn}\left(u^*_{x_N}(\boldsymbol{\rho}) - z_N\right) \end{pmatrix} : \boldsymbol{\rho} \in R_j \right\}\right| \leq ?$$

$\forall \boldsymbol{\rho} \in R_j$, duals are simultaneously structured: $u^*_{x_i}(\boldsymbol{\rho}) = g_i(\boldsymbol{\rho}), \forall i$

$$\left|\left\{\begin{pmatrix} \operatorname{sgn}(g_1(\boldsymbol{\rho}) - z_1) \\ \vdots \\ \operatorname{sgn}(g_N(\boldsymbol{\rho}) - z_N) \end{pmatrix} : \boldsymbol{\rho} \in R_j \right\}\right| \leq ?$$

Proof ideas

$$\left|\left\{\begin{pmatrix}\operatorname{sgn}\left(u^*_{x_1}(\boldsymbol{\rho}) - z_1\right)\\ \vdots \\ \operatorname{sgn}\left(u^*_{x_N}(\boldsymbol{\rho}) - z_N\right)\end{pmatrix} : \boldsymbol{\rho} \in R_j\right\}\right| \leq ?$$

$\forall \boldsymbol{\rho} \in R_j$, duals are simultaneously structured: $u^*_{x_i}(\boldsymbol{\rho}) = g_i(\boldsymbol{\rho}), \forall i$

$$\left|\left\{\begin{pmatrix}\operatorname{sgn}\left(g_1(\boldsymbol{\rho}) - z_1\right)\\ \vdots \\ \operatorname{sgn}\left(g_N(\boldsymbol{\rho}) - z_N\right)\end{pmatrix} : \boldsymbol{\rho} \in R_j\right\}\right| \leq \underline{(eN)^{\mathrm{Pdim}(\mathcal{G}^*)}}$$

Follows from key lemma

Proof ideas

$$\left|\left\{\begin{pmatrix} \operatorname{sgn}\left(u^*_{x_1}(\boldsymbol{\rho}) - z_1\right) \\ \vdots \\ \operatorname{sgn}\left(u^*_{x_N}(\boldsymbol{\rho}) - z_N\right) \end{pmatrix} : \boldsymbol{\rho} \in \mathbb{R}^d \right\}\right|$$

$$\leq \underbrace{(eNk)^{\mathrm{VCdim}(\mathcal{F}^*)}}_{\text{Number of regions}} \underbrace{(eN)^{\mathrm{Pdim}(\mathcal{G}^*)}}_{\text{Number of sign patterns within each region}}$$

$\mathrm{Pdim}(\mathcal{U})$ equals largest N s.t. $2^{\mathrm{N}} \leq (eNk)^{\mathrm{VCdim}(\mathcal{F}^*)}(eN)^{\mathrm{Pdim}(\mathcal{G}^*)}$,

so $\mathrm{Pdim}(\mathcal{U}) = \tilde{O}\left((\mathrm{VCdim}(\mathcal{F}^*) + \mathrm{Pdim}(\mathcal{G}^*)) \log k\right)$

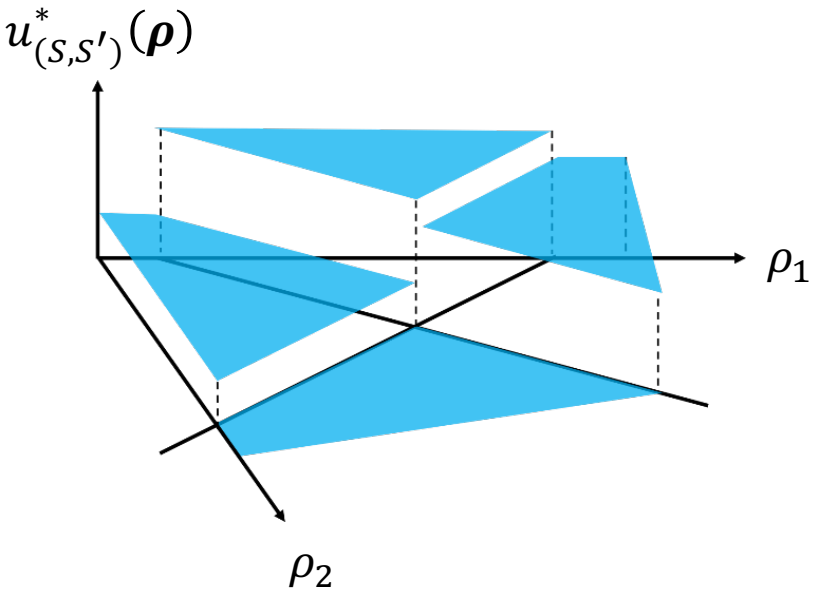
Outline

1. Introduction
2. Model and problem formulation
3. Our guarantees
   a. Example of piecewise-structured utility function
   b. Piecewise-structured functions more formally
   c. Main theorem
   d. Application: Sequence alignment
4. Conclusion and future directions

Piecewise constant dual functions

Lemma:

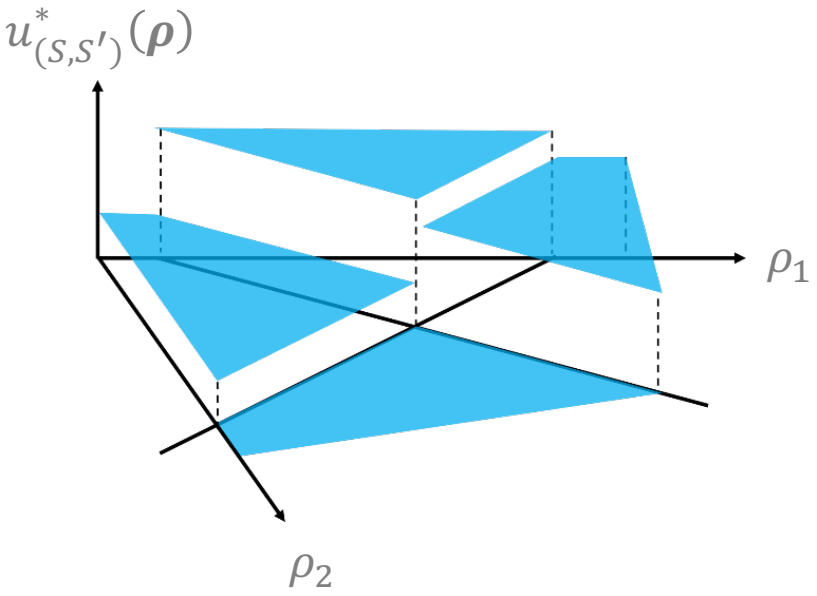
Utility is piecewise constant function of parameters



Sequence alignment guarantees

Theorem: Training set of size $\tilde{O}\left(\frac{\log(\text{seq. length})}{\epsilon^2}\right)$ implies for any $\boldsymbol{\rho}$,

Average utility over training set is ϵ-close to **expected** utility



Outline

1. Introduction
2. Model and problem formulation
3. Our guarantees
4. **Conclusion and future directions**

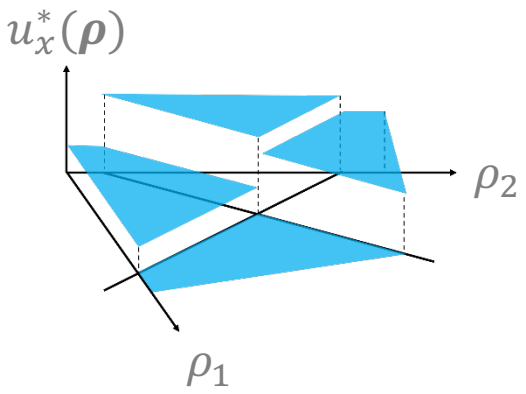
Conclusion

Guarantees for learning high-performing algorithm parameters

- Apply if performance is piecewise-structured function of parameters
- Proved by exploiting connections between primal and dual classes

Algorithm families from diverse domains exhibit this structure

- Clustering
- Economics (mechanism design)
- Integer programming
- Computational biology



Future research: Algorithms

This talk: Generalization guarantees

Apply to any configuration procedure (approximate, heuristic, optimal)

How to quickly find **provably** good parameters?

Growing body of work, but still many open questions

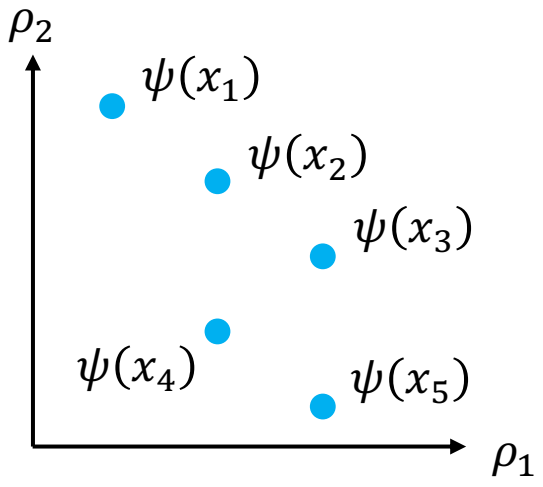
Kleinberg, Leyton-Brown, Lucier	IJCAI'17
Weisz, György, Szepesvári	ICML '18, '19
Kleinberg, Leyton-Brown, Lucier, Graham	NeurIPS'19

Future research: Mapping to parameters

This talk: Learn **1** parameter setting that's good in expectation

Future research: **More flexible approach**

Learn **mapping** from inputs to parameter settings



Future research: Data-dependent bound

Strength of our results: **Input-distribution agnostic**

Apply to **any** distribution over instances

Can be loose for non-worst-case distributions

Guarantees that improve based on "niceness" of distribution?

- Covering-style analysis
- Rademacher complexity

How much data is sufficient to learn high-performing algorithms?

Generalization guarantees for data-driven algorithm design

Ellen Vitercik
Carnegie Mellon University

Columbia Theory Seminar

Nina Balcan

Dan DeBlasio

Travis Dick

Carl Kingsford

Tuomas Sandholm